

Working with AIDLC

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [working-with-aidlc.pdf](#)

This guide helps you get the most out of AI-DLC (AI-Driven Development Life Cycle). It covers how to interact effectively with the AI at each stage — from first prompt to working code.

Start with the basics in each section. The advanced tips are drawn from real workshop experience and address the patterns that teams found most useful once they got comfortable with the fundamentals.

Table of Contents

1. [General Rules](#)
 2. [Inception Phase](#)
 3. [Construction Phase](#)
 4. [Never Vibe Code](#)
-

1. General Rules

Asking Questions Without Changing Files

One of the most important habits to build early: **not every question should trigger a document update.**

When you ask the AI something without guarding your question, it may interpret it as a change request and immediately update design documents. To prevent this, prefix exploratory questions with a clear no-change instruction.

Basic pattern:

Do not update any documents. Help me understand why [this decision] was made.

Do not update any documents. For [component name], is it reasonable to use [library or technology] here?

Do not change anything. Assess the impact of [proposed change].
I want to understand the consequences before we decide.

These patterns let you think out loud with the AI, evaluate options, and challenge decisions without committing to anything. Once you're satisfied with the answer, follow up with a deliberate update instruction if needed.

Tip: Start every exploratory message with "Do not update any documents." You can always drop that constraint once you're ready to act.

The Question → Doc → Approval Flow

AIDLC never asks clarifying questions inline in the chat. It writes questions into a markdown file and waits for you to fill in your answers there. This keeps a durable record of every decision and makes it easy for the whole team to contribute.

Step 1 — AIDLC creates a question file

The AI creates a file like `aidlc-docs/inception/requirements/requirement-verification-questions.md` and stops. It will not proceed until you answer.

Step 2 — You fill in your answers

Open the file and fill in each `[Answer]:` tag. Questions use multiple-choice format:

```
## Question: Deployment model
Where will this service be deployed?

A) AWS Lambda (serverless)

B) AWS ECS Fargate (containerized)

C) Existing on-premises infrastructure

X) Other (please describe after [Answer]: tag below)

[Answer]: B
```

A few things that work well when answering:

- **Add a label alongside the letter.** `C – financial summary and debt service coverage` is clearer than just `C`.
- **Include a brief justification.** `A – design-first; generate the OpenAPI spec before writing code` confirms intent and gives the AI context it carries forward.
- **Combine options when you mean both.** `B and C – rate limiting at both API Gateway level and application level (not D)` is unambiguous.
- **Add a caveat when the option is almost right.** `B – migration is a separate project; however, include a one-time migration into the new data structures.`
- **Use X freely.** If none of the options fit, X is the right choice over forcing a wrong answer.

Step 3 — Tell the AI your answers are ready

Return to the chat and say: "We have answered your clarification questions. Please re-read the file and proceed."

Tip: explicitly asking the AI to *re-read* the file ensures it loads your answers from disk rather than relying on an in-memory version that may not reflect your latest edits.

Step 4 — AIDLC validates and proceeds

The AI reads your answers, flags any remaining ambiguities, and proceeds to generate the next artifact.

Advanced tip: If you have documentation that answers some of the AI's questions, you can instruct it to resolve those itself: "Analyze the rationale for each question. If a question has already been answered through the provided documentation, answer it

yourself. Only ask me if it is still unclear." This reduces unnecessary back-and-forth at gate points.

Approval gates

At the end of each stage, AIDLC presents a completion message with two options:

- **Request Changes** — ask for modifications before moving on
- **Approve and Continue** — accept the output and advance

Read the generated artifact before approving. Discuss with your team if needed. Only approve when you're satisfied.

Context Management

Context is the AI's working memory for the session. AIDLC depends on having the full chain of artifacts and instructions in context to generate consistent downstream outputs. Managing it well is one of the highest-leverage habits you can develop.

The core rule: clear the context at every natural decision point.

AIDLC is built around gates — moments where the AI stops and asks you something: a question file to answer, a document to approve, a plan to review. These pauses are not just approval checkpoints. They are the right moments to start a fresh context before continuing.

Clearing context at a gate is low-risk because the AI's current work is already saved to files. The next context starts clean, loads the relevant artifacts from disk, and proceeds without carrying accumulated noise from all the earlier steps.

If you let context accumulate across multiple gates, the AI starts working from a compressed or partially lost version of earlier instructions and artifacts. Output quality degrades in ways that are subtle and hard to diagnose.

In practice:

- When the AI asks you to answer a question file — answer the questions, then **start a fresh context** and tell the AI to re-read the file and continue
- When the AI presents a document for approval — review it, then **start a fresh context** to either request changes or approve and proceed

- If your tool offers a "compact context" prompt mid-workflow, **always decline it** — compaction is not the same as a clean reset and loses more than it saves

How to resume after a context reset:

Option 1 — State file method (recommended):

```
Go to aidlc-docs/aidlc-state.md, find the first unchecked item,  
then go to the corresponding plan file and resume from that point.
```

Option 2 — Manual handoff:

```
I am resuming a previously stopped conversation. Here is the context:  
[paste summary of last output or recent change]  
Please continue with [next action or section X].
```

Tip: Commit and push all current changes to the repository whenever you reset context. It takes seconds and means you always have a clean recovery point.

```
Please commit and push all current changes to the repository.
```

Batching Prompts

Not all prompts should be sent separately. A simple rule from workshop experience:

When two changes are tightly coupled to the same subject, include both in one prompt. When two changes are unrelated, do them one at a time.

Over-batching (combining unrelated changes) causes the AI to lose focus and miss details. Under-batching (separate prompts for closely related things) adds unnecessary round-trips. When in doubt, err on the side of separating.

Loading External Reference Files

You can point AIDLC to any existing document — a schema, an architecture diagram, a data dictionary, an API spec — and it will incorporate that content into the current stage.

Basic pattern:

```
Please read [file path or description]. Use it as the basis for [what you want].
```

```
We have an existing audit table structure. Please add it to the inception documents and reference it for this service. When we proceed, expect new requirements and stories related to this service.
```

Advanced tip: You can load documents at any stage, not just at the start. If a new constraint surfaces during Construction — an updated security policy, a revised data model — load it and ask AIDLC to assess the impact before proceeding.

Advanced tip — Enterprise standards as extensions: If your organization has security, compliance, or API guidelines that should apply to every project, add them as a markdown steering file in `aidlc-rules/extensions/`. AIDLC will automatically load them into every phase without requiring manual injection.

Getting Independent Critiques

AIDLC will defend its own prior decisions. When you want an unbiased evaluation of an artifact, ask for a critique in a **fresh context** — one where the AI has no memory of why it made those decisions.

```
Produce a critique document of [the requirements document / the component design].  
Do this in a new context separate from everything else.
```

This produces more useful, objective feedback than asking for a critique in the same session where the artifact was created.

Depth Levels

AIDLC adapts how deeply it executes each stage based on the complexity of your request. You can influence this.

```
Keep this at minimal depth — we just need the basic structure documented.
```

2. Inception Phase

The Inception phase is where you and the AI align on *what to build and why* before any design or code work begins. The more context you bring in here, the fewer clarifying questions and the less rework you'll encounter in Construction.

Prepare Your Inputs Before Starting

The single most effective thing you can do before kicking off AIDLC is prepare two documents:

1. **Vision Document** — what to build and why
2. **Technical Environment Document** — what tools and constraints apply

These documents dramatically reduce the number of clarifying questions AIDLC will ask and ensure the AI starts from your team's actual context rather than making assumptions.

Where to start:

- [writing-inputs/inputs-quickstart.md](#) — quick summary for both greenfield and brownfield
- [writing-inputs/vision-document-guide.md](#) — full vision guide with templates
- [writing-inputs/technical-environment-guide.md](#) — full technical environment guide with templates

Brownfield projects (adding to an existing codebase) need slightly different inputs. The vision doc needs a current state description and an explicit list of what must not change. The technical environment doc should describe the existing stack rather than a desired one, and example code should come from actual existing files. See [writing-inputs/inputs-quickstart.md](#) for the brownfield minimum and worked examples.

Minimum viable input if you want to start quickly:

For the Vision: one paragraph describing what you're building and for whom, a list of MVP features in scope, a list of features explicitly out of scope, and any open questions — things you already know are uncertain. Open questions feed directly into Requirements

Analysis as pre-declared ambiguities, so they get resolved early rather than surfacing as surprises mid-design.

For the Technical Environment: language and version, package manager, web framework, cloud provider and deployment model, test framework, a prohibited libraries table (with reason and recommended alternative for each entry), security basics, and at least one example each for a typical endpoint, function, and test.

The prohibited libraries table matters more than a plain list — the reason and alternative columns tell AI-DLC *why* a library is banned, which leads to better substitution decisions. The example code patterns are the single highest-leverage addition beyond the basics: they give AI-DLC a concrete pattern to follow during code generation rather than inventing its own.

Tip: Every gap you fill in up front is one fewer clarifying question during Requirements Analysis.

Kicking Off a New Project

Once your input documents are ready:

```
I want to start a new project. Please read [path to vision document] and
[path to technical environment document], then begin the AIDLC workflow.
```

AIDLC will scan the workspace, determine greenfield vs. brownfield, and proceed into Requirements Analysis using your documents as the primary source — asking only for what they don't cover.

For a brownfield project, AIDLC will first run Reverse Engineering, analyzing your existing codebase and producing architecture, component, and API documentation. Review these artifacts carefully — they become the foundation for everything that follows.

Answering Requirements Questions

See the answering tips in [Section 1](#) for the full guidance on using letters, adding labels, combining options, and using X for custom answers. A few additional points specific to Requirements Analysis:

- **Separate the full vision from the MVP explicitly.** If AIDLC asks what features to include, name them. If something is out of scope, say so — don't leave it ambiguous.
- **State deliberate "no" decisions clearly.** `D – no caching required at this time` signals intent. An empty answer invites the AI to make a speculative choice.
- **Describe phased approaches inline.** `X – simple role-based workflow now; replace with external workflow engine when available` lets AIDLC design the current solution with the right extension points.

Advanced tip — Security Extensions: During Requirements Analysis, AIDLC will ask whether you want to enforce security extension rules. For production-grade applications, choose Yes. For prototypes, No is fine. This decision is recorded and enforced throughout Construction, so choose deliberately.

Inception-Specific Interactions

Deferring a feature mid-stream:

```
We are going to backlog the [feature name] capability for the current release.
Please remove it from the component design and flag the related user stories as backlogged.
```

Backlogging (rather than deleting) preserves the work for future iterations without it influencing the current build.

Registering an existing data structure:

```
We have an existing [schema/structure name]. Please add it to the inception documents
and reference it for this service. When we proceed, expect new requirements and
stories related to this service.
```

Making implicit data sources explicit:

```
For the [service name], add the understanding that [new data source] is also a
data source for this feature, in addition to [existing data source]. Then review
requirements and user stories to ensure this is captured.
```

Checking for upstream impact after a design change:

After any meaningful change to a design artifact, ask AIDLC to check whether earlier documents are still consistent:

```
Now review the previous steps – user stories and requirements – to ensure
this change does not require updates to any of those documents.
```

Advanced tip — Standing back-propagation rule: Instead of asking after each change, set this as a standing instruction at the start of a phase: "Every time you update a document, check whether the change impacts the requirements document and user stories, and prompt me if it does." This creates an automatic safety net without requiring you to remember.

Parallel team review of component design:

If your team splits up to review different components simultaneously:

```
Restrict your edits to the files under your team's control. When all teams are done,
we will ask the AI to review all changes and confirm there are no conflicts.
Then we will ask it to review impacts to user stories and requirements.
```

When everyone is done, trigger the conflict check:

```
We had [N] independent groups editing component design files. Please review all files
and report any conflicts or inconsistencies. Do not edit the files – produce a report
for our review.
```

Resolve each conflict explicitly by number:

```
For conflict #[number] ([conflict description]):
update [target file] to reflect [your decision].
```

```
For conflict #[number] ([capability name]):
this capability is backlogged. Update the documentation to clearly mark it as
backlogged so code generation does not attempt to implement it.
```

Archiving stale design files:

If exploration during design produced files that are no longer needed:

Move the [file descriptions] to an archive folder — do not delete them.
Then confirm whether they are required for code generation.

Advanced tip — Component size constraints: If you want to prevent oversized components that would be too large to implement in a single sprint, set a story-point cap during Application Design: "At the component design phase, inject the following instruction: no single component should have more than [X] aggregate story points. If a component exceeds this limit, break it down into smaller sub-components."

Advanced tip — Context resets mid-phase: If your session gets interrupted, use this to re-establish state:

```
text Stop. New context. We just completed [description of recent work]. Please review  
[upstream artifacts] to assess any impact of the recent change. [Paste the change  
description here.]
```

3. Construction Phase

The Construction phase is where designs become code. Each unit of work goes through a series of design stages (conditional) followed by Code Generation (always). After all units are complete, Build and Test closes out the work.

The Design Review Process

For each unit of work, AIDLC may execute some or all of these design stages before generating code:

- **Functional Design** — business logic, domain models, data schemas
- **NFR Requirements** — performance, security, scalability, tech stack selection
- **NFR Design** — applying NFR patterns to the design
- **Infrastructure Design** — mapping the design to actual cloud services

Each stage produces a document in `aidlc-docs/construction/{unit-name}/`. Your job at each gate is to read the document and decide: request changes or approve.

Read before you approve. The design documents are the source of truth for code generation. Mistakes that slip through here are harder to fix later.

Advancing from design to code:

When you're ready to transition to Code Generation, give the AI the structural context it needs up front:

```
We have completed component design review. We are ready for code creation.  
Please use the following directory and source code structure:  
[reference an existing service or folder structure].  
Use this pattern for APIs. For the UI, follow the [Vue.js composables/components/store]  
directory structure. Please ask any questions you have before proceeding.
```

Inviting questions before generation starts resolves ambiguities in the plan rather than in the middle of file creation.

Requesting a targeted correction:

Be precise — name the element, what is wrong, and what it should be:

```
The [endpoint description] should use [correct parameter], not [incorrect parameter].  
Please update the [component name] accordingly.
```

Choosing between AI-presented options:

```
Please implement Option B — [option description] — for [feature name].  
Update all component design documents accordingly.
```

Reference the option by letter *and* description, and explicitly scope the update to all affected documents, not just the one where the question arose.

Overriding a design pattern:

```
We prefer to deviate from [standard pattern] and use [our preferred approach]  
to allow [rationale]. Please update the component design documents accordingly.
```

The rationale matters. AIDLC carries it forward into later stages, which prevents the deviation from being silently reversed.

Advanced tip — Impact assessment before committing: For any significant design change, assess before acting:

```
text Do not change anything. Assess the impact of [proposed change]. [Describe the proposed change in detail.]
```

Advanced tip — Inline code documentation: If you want inline documentation applied consistently to every unit, add it as a standing rule at the start of the Construction phase rather than repeating it per unit: "Add inline code documentation as a standard rule for the construction phase."

The Code Generation Process

Code Generation has two distinct parts. Both require your explicit approval.

Part 1 — Planning

AIDLC creates a numbered, checkbox-tracked plan of every file to be created or modified. Review this plan before approving. Check that:

- Every file is in the right location (application code in the workspace root, never in `aidlc-docs/`)
- The steps cover everything your design documents specified
- Brownfield projects list existing files to modify, not new duplicates alongside them

Advanced tip — Internal libraries: Before approving the plan, inject your internal library requirements into the Q&A file or implementation plan:

```
text In addition to my answers, you must use the following libraries from our [starter project / building blocks]: [list each library explicitly]. Explain why and when each should be used, not just what it is.
```

A curated markdown guide to your internal libraries works better than pointing the AI at a repository. Create one and reference it as a code generation input.

Advanced tip — UI from Figma designs: Take a screenshot of your Figma design, pass it to a vision-capable model (e.g. ChatGPT) to generate framework code from the screenshot, then provide that output to AIDLC as the UI implementation input. This produces a concrete, tool-readable specification rather than a raw design-tool export.

Part 2 — Generation

AIDLC executes each step sequentially, checking off each step as it completes. When all steps are done, it presents the completion message with paths to generated files.

Review the generated code before approving. If something isn't right:

```
Request Changes: [describe specifically what needs to change]
```

Advanced tip — Brownfield file modifications: For existing codebases, AIDLC modifies files in place. If you see `ClassName_modified.java` or `service_new.ts` alongside the original, flag it immediately:

```
text I see [ClassName_modified.java] alongside [ClassName.java]. Please merge the changes into the original file and delete the duplicate.
```

Build and Test

After all units are complete, AIDLC generates build and test instructions for all units. A few patterns worth knowing:

Injecting test tooling at the right moment:

Don't add test framework or test management system instructions at project start. By the time code generation begins, those details may have been compressed or lost across many intervening stages. Inject them just-in-time:

```
At the functional test generation step, inject the following instruction:  
generate functional tests using the [test management system] format described  
in this document: [attach specification]. Use this API endpoint to push the  
generated test cases to the [test management system] repository: [endpoint details].
```

This principle applies to any tool-specific instruction: inject it at the phase where it's needed, not at project start.

Scoping unit test coverage:

```
When generating unit tests, exclude third-party external dependencies from  
code coverage calculations. Require a minimum of 80% coverage on internal  
code paths only.
```

After Code Generation: Back-Propagating Changes

Changes made during code generation — small design decisions, adjustments discovered while writing code — need to flow back up to the design documents. Do this as a deliberate sweep after code polish is complete, not ad hoc:

```
When you have finished polishing the code, review each unit's final design files  
and propagate any changes back up the chain to requirements and user stories.  
Make a plan for how to do this step by step before executing.
```

Asking for a plan before execution ensures the sweep is systematic across all units rather than selective.

Advanced tip — Extracting reusable specs: At the end of a completed project, extract the patterns you established into reusable specification documents for future projects:

```
text Create a set of reusable specification documents from the patterns expressed in this  
project: one for API design, one for security, one for UI specifications, one for the  
technology stack, and one for directory structure. Use the completed units as the source. I  
will review and approve each document before it is used in future projects.
```

4. Never Vibe Code

Vibe coding means directly editing generated code files to make quick fixes or try things out — bypassing the design documents entirely. It feels fast in the moment and creates problems shortly after.

The issue isn't the edit itself. It's that the design documents — the source of truth AIDLC uses for every subsequent operation — no longer reflect what the code actually does. The next time AIDLC runs Code Generation for a related unit, or you resume a session, or a colleague picks up the work, the disconnect causes confusion and rework.

One team described it directly during workshops:

"You never fix code directly. If you discover an issue, go back to AIDLC and say: I have discovered issue X. Review the design and make a plan to fix it. If this affects the design, update it, then update the code."

The rule: update the design first, then generate the code.

The Right Way to Make a Change

Whether you've spotted a bug, changed your mind about a design decision, or received new requirements, the flow is the same:

Step 1 — Describe the issue without touching anything:

```
Do not update any documents yet. I have discovered issue [X].  
Review the design and help me understand where this needs to be addressed.
```

Step 2 — Fix the design document:

```
Please update [specific design document] to reflect [the fix].  
Then check whether any upstream documents – requirements, user stories –  
also need to be updated.
```

Step 3 — Regenerate the affected code:

```
The design for [unit name] has been updated. Please re-run code generation  
for the affected files only.
```

This flow takes a few extra minutes compared to directly editing a file. It keeps your documentation in sync, your audit trail complete, and your team aligned on what was actually built.

When You're Tempted to "Just Edit the File"

"It's just a one-line fix."

One-line fixes that bypass the design still create drift. Note the fix in the relevant design document and let AIDLC apply it:

```
In [functional-design.md for unit X], update [method or rule] to [the fix].  
Then regenerate [the affected file].
```

"We're just exploring — nothing is final yet."

Exploration is exactly what "Do not update any documents" is for. Explore freely in the chat. Commit only when you're ready.

"I need to unblock the team right now."

Sometimes you have to move fast. If you make a direct edit, log it honestly so the audit trail stays accurate:

```
We made a temporary direct edit to [file] to unblock the team.  
The fix was [description]. Please update [design document] to reflect this  
and verify no other documents are inconsistent.
```

Standing Rules That Prevent Drift

Two standing instructions you can set at the start of a Construction phase that catch problems early, without requiring you to remember to ask each time:

Back-propagation on every update:

```
Every time you update a document, check whether the change impacts the  
requirements document and user stories, and prompt me if it does.
```

Design-first on every code decision:

```
When you make a design decision during code generation, always make sure  
the documentation reflects this change before proceeding.
```

Set these once at the start of Construction and they apply for the entire phase.

Keeping Reports Out of aidlc-docs

One practical note: if you ask AIDLC to produce human-facing reports — architecture diagrams, component summaries, stakeholder presentations — don't let it save them into `aidlc-docs/`. Those files will be loaded as artifacts in subsequent stages, inflating the token count and potentially confusing the AI about what is authoritative design input.

Use a separate `reports/` folder and, for cleaner output, generate reports in a fresh context with a dedicated report specification file:

```
Pause the process. Start a new context. Read [report specification markdown file]
and produce the report based on the current state of the AIDLC artifacts.
Save the output to a reports/ folder, not aidlc-docs/.
```

For guides on preparing your input documents, see [writing-inputs/inputs-quickstart.md](https://github.com/aidlc/aidlc/blob/main/docs/writing-inputs/inputs-quickstart.md).